

From Cap To Stalk: Predicting Mushroom Toxicity with ML Models

Kevin Nevarez
Computer Science Department
Cal Poly Pomona
Pomona, USA

Gerardo Gutierrez
Computer Science Department
Cal Poly Pomona
Pomona, USA

Brian Nguyen
Computer Science Department
Cal Poly Pomona
Pomona, USA

Vinson Liu
Computer Science Department
Cal Poly Pomona
Pomona, USA

Chelsea Kathleen Ocampo
Computer Science Department
Cal Poly Pomona
Pomona, USA

Abstract—Accurately identifying mushroom toxicity is important for public safety, agricultural decision-making, and regulatory oversight. Machine learning provides a scalable approach for determining which physical characteristics are most informative in distinguishing poisonous mushrooms from edible ones. In this study, we evaluate several classification algorithms, including Support Vector Machines, K-Nearest Neighbors, Random Forests, Logistic Regression, Naive Bayes, and Decision Trees, to predict mushroom toxicity and identify influential features. Using a simulated dataset of 61,069 mushroom instances derived from 173 species, we compare model performance and analyze feature contributions. Our findings highlight characteristics such as stem width, gill attachment, and gill color as potential key indicators of toxicity. While the dataset is simulated rather than field-collected, this work establishes a foundation for future studies that incorporate real-world biological data to improve practical applicability and public guidance.

Index Terms—Classification algorithms, Support vector machines

I. INTRODUCTION

Accurate classification of mushroom toxicity has practical implications for the general public, agricultural stakeholders, and regulatory agencies. For individuals who forage or consume wild mushrooms, improved guidance can reduce the risk of accidental poisoning. In agricultural and ecological workflows, identifying species characteristics associated with toxicity can support monitoring efforts and inform policy decisions. This project aims to use machine learning to classify mushrooms as edible or poisonous based on their physical features and to determine which characteristics contribute most to toxicity prediction.

The dataset used in this study is the Poison Mushroom Dataset provided by the University of California, Irvine Machine Learning Repository. The data consists of 61,069 simulated mushroom instances modeled after 173 species described in the reference text *Mushrooms and Toadstools*. Each instance includes 20 physical features, and mushrooms are labeled as either edible or poisonous. Species with uncertain or ambiguous edibility are conservatively categorized as poisonous to reflect real-world safety practices.

II. RELATED WORKS

In recent years, many studies have been performed in advance of determining the toxicity of mushrooms.

Creating the data set we have, Wagner et. al [2] utilizes a similar previous data set as inspiration and using references from primary textbooks about mushrooms to design and simulate instances of mushrooms. In the same paper, they used machine learning models such as Gaussian Naive Bayes, Random Forest, Linear Discriminant Analysis and Logistic Regression to analyze the newly created data set. The results of their paper are quite similar to ours: Random Forest being the most accurate model, and Naive Bayes being the least.

Comparatively, Nithya et. al [3] as well as Morshed et al [4] makes use of many different models to work on the same data set. Their preprocessing were similar than theirs in the way they handled the missing values but they did not specify the encoding method they used. The results they had similarly was that their top performers were KNN, and Linear SVM.

We build on their work with putting most of the improvement on preprocessing. By creating our own KNN imputer and standardizing the data, the data is more tailored to be fed to the models. Not only did we get similarly accurate results from their top models (RF, KNN, SVM) but we also were able to model another accurate classifier (Decision Tree) different from their results.

III. DATASET DETAILS

The data set for this project is a tabular dataset of simulated mushrooms and toadstools from the University of California Irvine Machine Learning Repository [1]. The features of the dataset include mushroom species, physical features of each part of the mushroom, habitat and season growth information, etc. The target or class of the data set specifies if a specific mushroom instance is edible or poisonous (binary classification).

The data set contains 61,068 instances of mushrooms, with 20 different features for each instance - 17 categorical and 3 numeric. There are 173 different species of mushrooms present



Fig. 1. Class Distribution

in the data set. The class distribution of the data set is as follows: (1) Poisonous - 33,888 instances (55.49%), (2) Edible - 27,181 instances (44.51%).

A. Preprocessing

1) *Imputation*: We dropped the following features with over 50% missing values: *stem root*, *veil type*, *veil color*, *stem surface*, *spore print color*. Then we split the data into 80/20 train and test splits before imputation to prevent data leakage.

For imputation, we chose to use KNN imputation as opposed to a simple imputation methods such as mode imputation. This was to get a more accurate representation of the missing data.

We created a custom k-nearest neighbors imputer which locally standardized numeric columns and one hot encoded the categorical columns before using k=5 nearest neighbors to perform KNN imputation. Only categorical features were missing values so our KNN imputer used majority voting to predict the missing values.

The remaining columns of the training data with no missing values were fitted into the KNN imputer. Then, the same fitted imputer was used to impute the remaining columns with missing values in both the training and test data.

2) *Standardization*: We then used Sklearn's StandardScaler to standardize numeric features. We fit a standard scaler to the training data and standardized the numeric values in both the training data and test data using the same fitted scaler. Standardization helps models like KNN and SVM, that use distance, to be less sensitive to noise in numeric features.

3) *Encoding Categorical Features*: The categorical features were encoded using Sklearn's OneHotEncoder.

IV. METHODOLOGY

We implemented multiple machine learning models to classify mushrooms as edible or poisonous. All models were trained using the preprocessed and encoded dataset described in the previous section. The models selected for this study include K-Nearest Neighbors (KNN), Naive Bayes, Decision Trees, Random Forests, Support Vector Machines (SVM), and

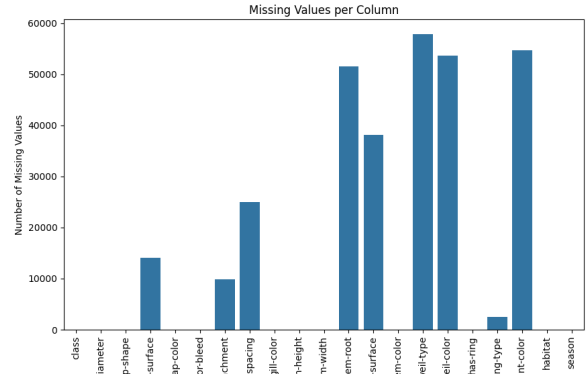


Fig. 2. Missing Values Per Feature

Logistic Regression. These models were chosen to represent a diverse set of classification approaches, including linear, nonlinear, distance-based, probabilistic, and ensemble methods. Model performance was later evaluated using confusion matrices and the following classification metrics: precision, recall, F1-score, and accuracy.

A. Logistic Regression

A Logistic Regression classifier was trained to provide a linear baseline model for comparison. We used an ℓ_2 -regularization to reduce overfitting with the regularization hyperparameter $C=1.0$.

B. Decision Tree

A Decision Tree classifier was trained using the one-hot encoded features. Decision Trees can capture nonlinear relationships and interactions between categorical attributes. To identify appropriate hyperparameters and reduce the risk of overfitting, we performed a grid search with 5-fold cross-validation. The hyperparameter grid included: 'max_depth': [5, 10, 15, 20], 'min_samples_split': [2, 5, 10, 20], 'min_samples_leaf': [1, 2, 5, 10], 'criterion': ['entropy', 'gini']. This allowed us to choose a tree size and splitting strategy that balanced model complexity and generalization ability.

C. Naive Bayes

Because our dataset contains a mix of numeric and categorical features, we adopted a simpler approach for this classifier. Specifically, only the categorical features were used, and the three numeric features were removed. A Bernoulli Naive Bayes model was then trained on the one-hot encoded categorical data, since it aligns well with its requirement of binary features.

D. Support Vector Machines

1) *Linear SVM*: A linear Support Vector Machine was trained using hinge loss and a regularization parameter of $C = 1$. The linear SVM serves as another baseline linear classifier. The chosen regularization parameter allows misclassification to prevent overfitting

The optimal value of k is 1

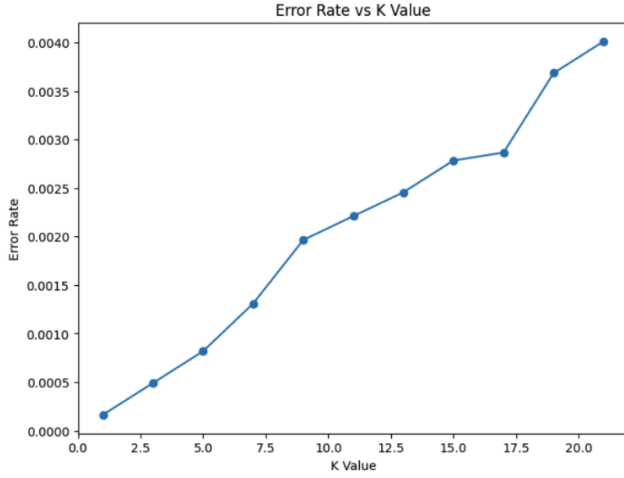


Fig. 3. Graph of KNN results

2) *RBF SVM*: We additionally trained an SVM with a radial basis function (RBF) kernel. The RBF kernel allows the classifier to learn more complex decision boundaries than the linear model. We used hinge loss and again set $C = 1$ to maintain consistency with the linear SVM.

E. KNN

For the K-Nearest Neighbors (KNN) algorithm, we first trained a model then evaluated the optimal k value that would minimize the error rate. This had the challenge of being computationally expensive. At first we planned to use grid search with five-fold cross-validation. Unfortunately this did not solve the computational expense problem. Because of this, we then decided to use the elbow method with a stopping criteria. The program would stop the evaluation if there was a consecutive decrease, i , in error rate as a result of ascending k values. The program additionally only tested odd values of k to avoid variance due to ties. At first, we used a large value of i to avoid potential local maxima in error rate. The results show that lower values of k resulted in a near-perfect accuracy measurement for KNN. In fact, using $i = 10$ yielded the graph shown in Fig. 3. With the model, $k=1$ was the optimal value for k as seen in Fig. 3.

F. Random Forest

Random forest was chosen for its ability at capturing feature importances and for how robust it is to noise and overfitting. For the model, we simply utilized a default Random Forest classifier with 100 trees ($n_estimators = 100$) and a fixed random state ($random_state = 42$) for reproducibility. As this configuration gave ideal results in the starting phase, we decided to simply keep the default hyper-parameters.

G. Feature Importance

We also decided to analyze the feature importance present in our top model, the Random Forest (RF) model. Feature

importance shows which features are contributing the most significantly towards predictions. There were two ways of extracting the feature importances within the RF model: (1) Mean Decrease in Impurity (MDI), and (2) Permutation. MDI offers a fast calculation but is biased towards high cardinality features (features with many unique values). In comparison, permutation is much more computationally heavy, but is more reliable and accurate as it does not suffer the same biases that MDI has. Because of this, the diagram that would be included in this paper would be with the permutation algorithm (Fig. 5).

V. RESULTS

We used the classification metrics, precision, recall, F1 Score, and accuracy for each model.

A. Decision Tree

The Decision Tree classifier achieved a precision of 0.9976, recall of 0.9965, and F1-score of 0.9971 for edible mushrooms, and a precision of 0.9972, recall of 0.9981, and F1-score of 0.9976 for poisonous mushrooms. The overall accuracy was 0.9974.

B. Naive Bayes

Naive Bayes obtained a precision, recall, and F1-score of 0.69 for edible mushrooms and 0.75 for poisonous mushrooms, resulting in an overall accuracy of 0.72.

C. Logistic Regression

Logistic Regression produced a precision of 0.75, recall of 0.76, and F1-score of 0.75 for edible mushrooms, and a precision of 0.80, recall of 0.79, and F1-score of 0.80 for poisonous mushrooms. The overall accuracy was 0.78.

D. Support Vector Machines

The Linear SVM achieved a precision, recall, and F1-score of 0.77 for edible mushrooms, and 0.81, 0.82, and 0.82 respectively for poisonous mushrooms, with an overall accuracy of 0.80. The RBF SVM achieved a precision, recall, and F1-score of 0.94 for edible mushrooms and 0.95 for poisonous mushrooms, with an overall accuracy of 0.95.

E. KNN

The KNN classifier achieved a precision of 1.0000, recall of 0.9998, and F1-score of 0.9999 for edible mushrooms, and a precision of 0.9997, recall of 1.0000, and F1-score of 0.9999 for poisonous mushrooms. The overall accuracy was 0.9999.

F. Random Forest

As seen in Table 1, the basic Random Forest (RF) model gave the most accurate predictions out of the many different models we tested. The Random Forest model achieved a recall of 0.9998, and a precision and F1-score of 1.0000 for edible mushrooms, and a precision of 0.9999, recall of 1.0000, and F1-score of 0.9999 for poisonous mushrooms. The overall accuracy was 0.9999. Of the 12214 instances, the RF model only predicted 1 instance wrong (Fig. 4).

TABLE I
ALL MODEL RESULTS

Model	Precision	Recall	F1-Score	Class	Support	Accuracy
Naive Bayes	0.69	0.69	0.69	Edible	5436	0.72
	0.75	0.75	0.75	Poisonous	6778	
Logistic Regression	0.75	0.76	0.75	Edible	5436	0.78
	0.80	0.79	0.80	Poisonous	6778	
Linear SVM	0.77	0.77	0.77	Edible	5436	0.80
	0.81	0.82	0.82	Poisonous	6778	
RBF SVM	0.94	0.94	0.94	Edible	5436	0.95
	0.95	0.95	0.95	Poisonous	6778	
Decision Tree	0.9976	0.9965	0.9971	Edible	5436	0.9974
	0.9972	0.9981	0.9976	Poisonous	6778	
KNN	1.0000	0.9996	0.9998	Edible	5436	0.9998
	0.9997	1.0000	0.9999	Poisonous	6778	
Random Forest	1.0000	0.9998	0.9999	Edible	5436	0.9999
	0.9999	1.0000	0.9999	Poisonous	6778	

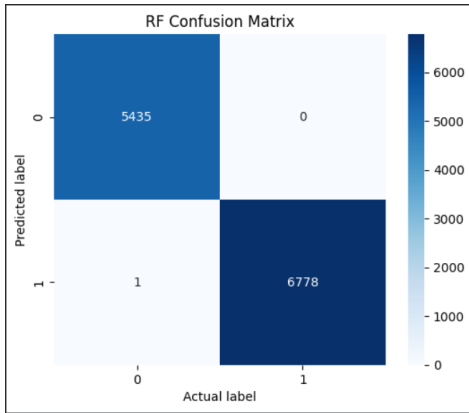


Fig. 4. Confusion Matrix for Random Forest Model

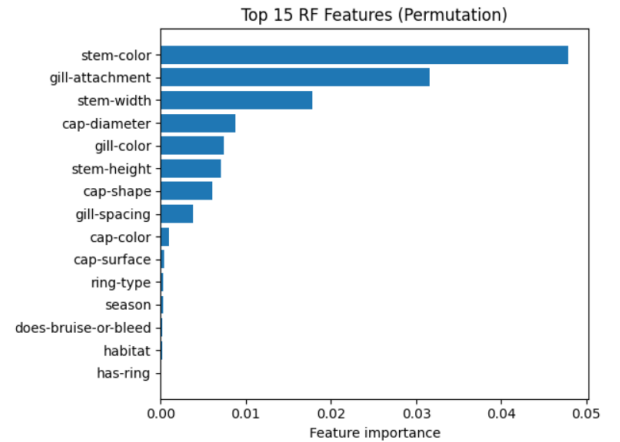


Fig. 5. Top features for the Random Forest model with the permutation algorithm

G. Feature Importance

As seen in Fig. 5, the top features that contribute to the model include ‘stem-color’, ‘gill-attachment’, and ‘stem-width’ while bottom features include ‘has-ring’, ‘habitat’, and ‘does-bruise-or-bleed’.

VI. DISCUSSION

In conclusion, random forests was shown to be the best model for classifying poisonous mushrooms with KNN and decision trees following closely with all three models having near perfect accuracy.

As seen within Fig 5., the most important features for predicting toxicity are stem color, gill attachment, and stem width. So these features should be considered more heavily by those working in agriculture and regulatory agencies like the FDA when creating public guidelines.

VII. SUPPLEMENTARY MATERIALS

Github Repository (contains LaTeX and source code): <https://github.com/GerardoG2/PoisonMushroomClassifier>

REFERENCES

- [1] Wagner, D., Heider, D., and Hattab, G. (2021). Secondary Mushroom [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5FP5Q>.
- [2] Wagner, D., Heider, D., and Hattab, G. (2021). Mushroom data creation, curation, and simulation to support classification tasks. Scientific Reports, 11.

- [3] Nithya, B., Asha, V., Sreeja, S.P., Khan, A., Mohapatra, A., and Patro, A. (2025). Edible Mushroom Classification with Machine Learning Techniques. 2025 5th International Conference on Pervasive Computing and Social Networking (ICPCSN), 266-271.
- [4] M. S. Morshed, F. Bin Ashraf, M. U. Islam and M. S. R. Shafi, "Predicting Mushroom Edibility with Effective Classification and Efficient Feature Selection Techniques," 2023 3rd International Conference on Robotics, Electrical and Signal Processing Techniques (ICREST), Dhaka, Bangladesh, 2023, pp. 1-5, doi: 10.1109/ICREST57604.2023.10070049.